Core Addon for Voxel Play - Documentation

Overview

The **Voxel Play Core Addon** is a free foundational framework designed to enhance the inventory system in **Voxel Play 3**. It introduces:

- **A hotbar system** for quick item access.
- **An extension of the base Voxel Play UI**
  Note: *Due to how the swap system works, Voxel Play's 'Build' function (where all items are unlocked like Minecraft Creative Mode) will not work correctly when this addon is active.*
- **A swap-based inventory system**, allowing easy item transfers and stacking.
- **A modular structure** that integrates smoothly with other **Incredible Extensions addons**.

This addon is essential for any Voxel Play-based project that requires a more interactive inventory experience.

Installation & Setup

Importing the Package

1. Open **Unity** and load your **Voxel Play 3** project.
2. Import the **Voxel Play Core Addon** package.
3. Ensure **Voxel Play 3** is already installed, as this addon depends on it.

Enabling the Core System

To set up the **Core Addon**, you need to:

Replace the default Voxel Play UI with the **Custom Game UI Canvas** provided in the addon.
Place the **FFGDsManagers** prefab into your scene

.

Creating Your Own UI (Optional)

Alternatively, you can **create your own custom canvas**, but if you want to reuse the provided **CustomGameUI** script, your canvas **must follow the exact structure of the Custom Game UI prefab**.

This is because **CustomGameUI** retrieves its references using the `CheckReferences()` method, which looks for specific **child objects by name**.

If you are creating your own canvas and **want to use the existing CustomGameUI script as-is**, make sure your **canvas includes these child objects** (with these exact names):

- **BaseInventory**
- **HotBar**
- **DragIcon**
- **PauseMenu**
- **Console**
- **Status**
- **ItemPlaceholder**
- **InitPanel**

Flexibility: Using Your Own Reference System

If you prefer a different way to assign references (for example, manual assignment in the inspector), you are **free to modify or remove the `CheckReferences()` method** inside CustomGameUI.
This lets you fully customize how your UI works, without being locked to the predefined hierarchy.

Recommendation

Before creating a custom canvas, it's recommended to **review the provided Custom Game UI Canvas** prefab to understand its structure and workflow.
This helps **avoid compatibility issues** when integrating with the Core Addon.

Replacing the Original UI

1. Locate the **Voxel Play UI Canvas** in your scene.
2. Delete or disable the default UI.
3. Drag and drop the **Custom UI Canvas** prefab from the Core\Prefabs into the scene.
4. Ensure the **Custom UI Canvas** is active and properly aligned in your game view.

Adding the FFGDsManagers

1. Locate the **FFGDsManagers** prefab inside the Core\Prefabs folder.
2. Drag and drop the prefab into your scene.
3. Ensure the managers are active in the **Hierarchy**.
4. Save your scene to apply changes.

This setup ensures that the **Core Addon's UI and inventory systems function properly.**

Hotbar System

The **hotbar system** is an integral part of the **Custom UI Canvas** and allows players to quickly access their items.

Enabling and Disabling the Hotbar

The hotbar can be toggled dynamically using the **ToggleHotBar** method:

```
CustomGameUI.Instance.ToggleHotBar(true); // Enable hotbar
CustomGameUI.Instance.ToggleHotBar(false); // Disable hotbar
```

This allows developers to control the visibility of the hotbar based on game state (e.g., showing it only when in survival mode).

Inventory System

The Core Addon uses the **default Voxel Play inventory system** but improves interaction through the **Inventory Swap Manager**.

Basic Interaction

- **Left Click** → Pick up the **entire stack** from a slot.
- **Right Click** → Pick up **half of the stack** from a slot.
- **Left Click (while holding an item)** → Place the **entire stack** into an empty or occupied slot.
- **Right Click (while holding an item)** → Place only **one** item from the stack into the slot.

These mechanics ensure smooth and intuitive inventory management for players.

Using the Swap Manager in Your Own Project

The **Inventory Swap Manager** is responsible for handling all **drag-and-drop item interactions** within inventories.

If you want to create your own custom inventory UI (instead of using the provided **CustomGameUI**), this section explains how to properly hook into the swap system.

How the Swap Manager Works

The **InventorySwapManager** provides a **central logic hub** for item selection, stacking, splitting, and swapping between slots.

This manager ensures consistent behavior across player inventories, external inventories (like chests, found in the External Inventories addon), and other addons.

Core Methods and Their Functions

- **SelectItem(IInventoryContainer container, int slotIndex, bool isPlayerSlot, bool isRightClick)**
  Handles **item selection and placement** when a slot is clicked.
  - **Left Click:** Picks up or places the full stack.
  - **Right Click:** Picks up or places a single item.
  - Automatically merges stacks if the same item is placed.
- **ClearSelection()**
  Clears the currently held item. This is called automatically when interactions finish but can also be triggered manually (e.g., when closing inventory).
- **OnItemSwappedEvent (Unity Event)**
  Fires after every successful swap.
  You can listen to this to trigger **UI refreshes, sound effects, or additional gameplay logic**.

Implementing Swap Logic in Your Inventory UI

To correctly hook into the swap system:

1. **Ensure every inventory slot uses InventorySlot or a subclass of InventorySlot.**
   This class handles the required communication with the Swap Manager.

2. **Ensure your UI updates properly when items are swapped.**
   For example, you might refresh icons or reapply tooltips.
   Listening to `OnItemSwappedEvent` can help trigger this refresh.

3. **(Optional)** Listen to `OnItemSwappedEvent` to trigger any additional behavior you want (like playing sound effects or highlighting affected slots).

Using this setup, your custom inventory UI can now **seamlessly swap and move items** using the **Voxel Play Core Addon's** Swap Manager.